汇编语言与接口技术

第4章 80X86汇编语言程序设计



主编：王让定 朱莹

宁波大学信息学院

本章主要内容



中断系统的功能

- MASM宏汇编语句结构以及开发过程
- MASM汇编语言表达式、运算符
- 程序段的定义和属性
- 复杂数据结构
- 宏汇编
- 常用的系统功能调用
- 汇编语言顺序程序设计

本章主要内容



中断系统的功能

- 汇编语言分支程序设计
- 汇编语言循环程序设计
- 串处理程序设计
- 子程序设计
- 高级汇编语言程序设计
- 汇编语言与C语言混合编程



MASM宏汇编语句结构以及开发过程

PART 01

汇编语言程序的语句类型和格式

1．语句的类型

- 硬指令语句(指令性语句)

  是指能产生目标代码，CPU可以执行的，能完成特定功能的语句，它主要由CPU指令组成。

- 伪指令语句（指示性语句）

  是一种不产生目标代码的语句，它仅仅在汇编过程中告诉汇编程序应如何汇编。

  - 定义变量，定义过程，给变量分配存储单元，给数字或表达式命名等。所以伪指令语句是汇编程序在汇编时用的，不产生机器码。

- 宏指令语句　它是一个指令序列，汇编时凡有宏指令语句的地方都将用相应的指令序列的目标代码插入。

指令性语句是计算机可执行的，而***伪指令***不能执行，用来定义变量，分配存储单元，指示程序开始和结束等。

这两种语句的差别：

在形式上：带不带**冒号，是否可转向它**.

在本质上：汇编时能否形成**对应的机器码**.

2. 语句的格式

- 汇编语言源程序由语句序列构成。
- 汇编语言源程序中的每条语句一般占一行，每行不超过132个字符（MASM 6.0开始可以是512个字符），
  - **执行性语句**
  - 标号：硬指令助记符　操作数，操作数；注释
  - **说明性语句**
  - 名字　伪指令助记符　参数，参数….　；注释

汇编语言的程序格式

- 在实地址模式和虚拟8086模式下，按照逻辑段组织程序，具有代码段、数据段、附加段和堆栈段。
- 一个汇编语言源程序可以包含若干个代码段、数据段、堆栈段或附加段，段与段之间的顺序可随意排列。
- 需独立运行的程序必须包含一个代码段，并指示程序执行的起始位置，一个程序只有一个起始位置。
- 所有的可执行性语句必须位于某一个代码段内，说明性语句可根据需要位于任一段内

- 两种格式书写
- 第一种格式是从MASM 5.0开始支持的简化段定义格式（但其中的两个指令 .STARTUP和 .EXIT是6.0版本才引入的）。
- 第二种格式是MASM 5.0以前版本就具有的完整段定义格式。

. **例** 简化段定义的源程序格式

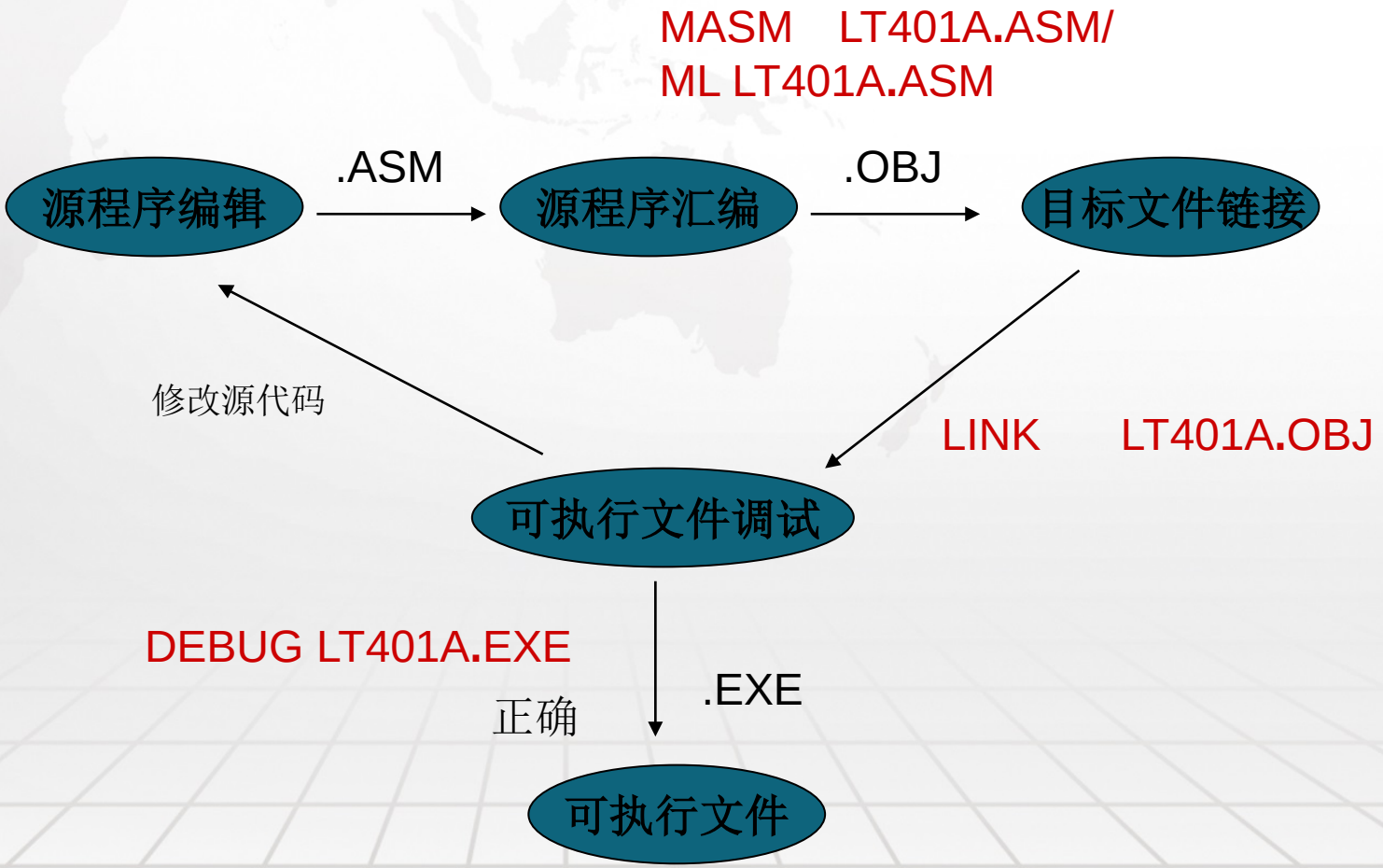
```
MODEL   SMALL   ; 定义程序的存储模式，小型程序一般采用小模式SMALL
    . STACK                         ; 定义堆栈段
    . DATA                          ; 定义数据段
     STRING     DB ’ Hello, Everybody !’ , 0DH, 0AH, ’ $’
       ; 在数据段定义要显示的字符串
    . CODE                          ; 定义代码段
    . STARTUP  ; 说明程序起始位置，并建立DS、SS内容（注1）
       MOV    DX, OFFSET STRING     ; 指定字符串在数据段的偏移地址
       MOV    AH, 9
       INT          21H     ; 利用DOS功能调用显示信息
    . EXIT 0                        ; 程序结束点，返回DOS（注2）
       END                                  ; 汇编结束（注3）
```

例 完整段定义的源程序格式

```
    STACK    SEGMENT STACK        ; 定义堆栈段STACK
      DW             512 DUP (?)          ; 堆栈段的大小是1024
  字节 (512字) 空间
    STACK    ENDS                          ; 堆栈段结束
    DATA     SEGMENT                       ; 定义数据段DATA
STRING       DB  'Hello, Everybody ! ', ODH, OAH, ' $'
; 在数据段定义要显示的字符串
    DATA     ENDS                          ; 数据段结束
    CODE     SEGMENT   'CODE'  ; 定义代码段CODE
      ASSUME  CS: CODE, DS: DATA, SS: STACK; 确定CS、DS、
  SS指向的逻辑段
    START: MOV AX, DATA       ; 设置数据段的段地址DS
           MOV DS, AX
```

```
MOV DX, OFFSET STRING
; 利用功能调用显示信息
MOV AH, 9
INT  21H
MOV AX, 4C00H ; 利用系统功能调用返回DOS
INT  21H
CODE  ENDS                          ; 代码段结束
END   START; 汇编结束，同时表明程序起始位置为标号
   START处
```

汇编语言程序的开发过程



- ML.EXE　　汇编程序
- ML.ERR　　汇编错误信息文件
- DOSXNT.EXE　　MS-DOS扩展文件
- LINK.EXE　　连接程序
- LIB.EXE　　子程序库管理文件
- MASM5.1以下:
  - MASM.EXE



MASM汇编语言表达式、运算符

PART 02

常量、运算符及表达式

1. 常量

(1)数字常量

二进制常量，以B结尾

十进制常量，以D结尾或省略(汇编语言中默认无标记数为十进制数)

十六进制常量，以H结尾，如0A8C6H.

MASM提供基数控制，.RADIX伪指令可以改变默认进制。

伪指令格式： .RADIX n

要求：n为2~16范围内任何数值。

功能：把n表示的数值作为默认基数。

例如指令“.RADIX 16”，将缺省基数改为16

(2)字符串常量

用**单引号**引起来的字符或字符串也代表常数。

例:’A’,’BCDE’,汇编时被翻译成对应的ASCII码41H和42H,43H,44H,45H。

字符串最大长度为255个字符

(3) 符号常量

利用一个标识符表达的一个数值。常数若使用有意义的符号名来表示，可以提高程序的可读性，同时更具有通用性。

MASM提供等价机制，用来为常量定义符号名，符号定义伪指令有“EQU”和“=”

2．运算符

MASM 6.x支持多种运算符，如表4.1所示。

（1）算术运算符

实现加、减、乘、除、取余的算术运算。其中MOD也称为取模，它产生除法之后的余数，如19 MOD 7 = 5。

```
MOV        AX，3 * 4+5 ； 等价于MOV        AX，17
```

加“ + ”和减“ - ”运算符还可以用于地址表达式。除加、减外，其它运算符的参数必须是整数。

（2） 逻辑运算符

实现按位相与、相或、异或、求反的逻辑运算。例如：

```
OR     AL，03H AND 45H          ； 等价于OR  AL，01H
```

（3） 移位运算符

实现对数值的左移、右移的逻辑操作，移入低位或高位的是0。

其格式为：SHL/SHR移位次数。例如：

```
MOV         AL，0101B  SHL（2*2）
            ；等价于MOV       AL，01010000B
```

逻辑和移位运算符与指令助记符相同，并有类似的运算功能。汇编程序能够根据上下文判断它们是指令还是运算符，前者进行代码翻译，后者汇编时计算其数值。

(4) 关系运算符

用于比较和测试符号数值，MASM用FFFFH（补码 -1）表示条件为真，用0000H表示条件为假。例如：

```
MOV    BX, ( (PORT LT 5)  AND 20)  OR  ( (PORT GE 5) AND 30)
  ; 当PORT<5时. 汇编结果为MOV BX, 20
    ; 否则，汇编结果为MOV BX, 30
```

(5) 高低分离符

取数值的高半部分或低半部分。

HIGH、LOW从一个字数值或符号常量中得到高、低字节，例如：

```
MOVAH, HIGH  8765H     ; 等价于MOV      AH, 87H
```

MASM 6.0引入的HIGHWORD, LOWWORD取一个符号常量（不是一般的常量）的高字或低字部分，例如：

```
DD_VALUE  EQU 0FFFF1234H; 定义一个符号常量
MOV AX, LOWWORD DD_VALUE     ; 等价于MOV AX, 1234H
```

运算符

运算符类型	运算符号及说明
算术运算符	+（加） -（减） *（乘） /（除） MOD（取余）
逻辑运算符	AND（与） OR（或） XOR（异或） NOT（非）
移位运算符	SHL（逻辑左移） SHR（逻辑右移）
关系运算符	EQ（相等） NE（不相等） GT（大于） LT（小于） GE（大于等于） LE（小于等于）
高低运算符	HIGH（高字节） LOW（低字节） HIGHWORD（高字） LOWWORD（低字）

3、运算符的优先级

优先级	运算符
1	（） <> [ ] • LENGTH SIZE WIDTH MASK
2	PTR OFFSET SEG TYPE THIS :
3	HIGH LOW
4	* / MOD SHL SHR
5	+ -
6	EQ NE GT LT GE LE
7	NOT
8	AND
9	OR XOR
10	SHORT

TYPE运算符用来取存储器的单元类型，各单元类型对应值如下：

存储器单元类型	对应值
DB(字节)	1
DW(字)	2
DD(双字)	4
NEAR	-1
FAR	-2

若BUFFER1存储区是用如下伪指令定义的：

```
BUFFER1    DB    100 DUP(?)
```

则 TYPE BUFFER1等于1

LENGTH运算符用来计算一个存储区的单元(单元可以是字节,字或双字)的数目。

注意:LENGTH返回的存储区必须用复制操作符DUP( )来定义,否则返回值为1。

例:

若FEES被定义为: FEES　DW　4,5,6

则 MOV　CX,LENGTH FEES

汇编后,MOV　CX,1

SIZE运算符用来计算一个存储区的字节总数。

例:

若BUFFER2存储区是用如下伪指令定义的:

```
BUFFER2   DW      200 DUP(0)
```

则:

```
TYPE      BUFFER2          等于2
LENGTH    BUFFER2          等于200
SIZE      BUFFER2          等于400
```

(6)合成运算符

用来建立或临时改变变量或标号的类型或存储器操作的存储单元类型。包括：PTR、THIS、SHORT。

PTR运算符

用来指定或修改存储器操作数的类型，通常和伪指令BYTE、WORD等连起来使用。

```
例：MOV     BYTE PTR[DI],0
    MOV     WORD PTR[DI],0
 而 MOV     [DI],0    ；类型不定
```

例: STRI1 DW ? ; 定义为字类型

可用如下语句:

```
PP    EQU   BYTE   PTR STRI1
QQ    EQU   BYTE   PTR STRI1+1
```

通过PP,QQ把STRI1和STRI1+1又规定为字节类型。

```
MOV   STRI1,AX          是合法的
MOV   AL,STRI1          是非法的
```

只能用: MOV AL,PP

或 MOV AL,BYTE PTR STRI1

THIS运算符

可以指定存储器操作数的类型。使用THIS运算符可以使标号或变量具有灵活性。

例:

```
AREAW EQU THIS WORD
AREAB DB 100 DUP(?)
```

SHORT运算符

可以指定一个标号的类型为SHORT(短标号)，即标号到引用该标号之间的距离在－127—＋127个字节范围内。短标号可以用于转移指令中，使用短标号的指令比使用近程标号的指令少一个字节。

(7) 其它运算符

1）方括号[ ]

指令中用方括号表示存储器操作数，方括号里的内容表示操作数的偏移地址。例如：

```
MOV  AX, [BX]        ；将（BX）和（BX+1）指向的存储器两个单元的内容送AX
```

2）段超越运算符“:”

运算符“:”（冒号）跟在某个段寄存器名（DS、ES、SS或CS）之后表示段超越，用来指定一个存储器操作数的段属性，而不管其原来隐含的段是什么。例如：MOV AX, ES:[DI] ;

把ES段中由DI指向的字操作数送(AX)

4. 表达式

表达式是常量、寄存器、标号、变量与一些运算符组合的序列，分数字表达式和地址表达式两种。汇编时按一定的优先规则对表达式进行计算后可得到一个数值或一个地址。

1．数字表达式（number expression）

数值表达式一般是指由运算符（MASM统称为操作符Operator）连接的各种常量所构成的表达式。汇编程序在汇编过程中计算表达式，最终得到一个数值。由于在程序运行之前，就已经计算出了表达式，所以，程序运行速度没有变慢，然而程序的可读性却增强了。

例如：MOV DX，(6*A-B)/2

指令的源操作数(6*A-B)/2是一个表达式。若设变量A的值为1，变量B的值为2，则此表达式的值为(6*1-2)/2 = 2，是一个数字结果，此表达式是数字表达式。

2．地址表达式（address expressions）

地址表达式的结果是一个存储单元的地址。当这个地址中存放的是数据时，称为变量；当这个地址中存放的是指令时，则称为标号。

当在指令的操作数部分用到地址表达式时，应注意其物理意义。例如，两个地址相乘或相除是无意义的，两个不同段的地址相加减也是无意义的。经常使用的是地址加减数字量。例如，SUM+1是指向SUM字节单元的下一个单元的地址。

又如：MOV AX, ES: [BX+SI+1000H]

BX+SI+1000H为地址表达式，结果是一个存储单元的地址。

变量及其属性

1．变量定义伪指令

变量是存储器中某个数据区的名字，在指令中可以作为存储器操作数。

变量定义（Define）伪指令可为变量申请固定长度的存储空间，并可以同时将相应的存储单元初始化。该类伪指令是最经常使用的伪指令。

伪指令格式：[变量名] 伪指令 初值表

说明：

- 变量名是用户自定义标识符，表示初值表首元素的逻辑地址，即用这个符号表示地址，常称为符号地址。变量名可以没有，这种情况，汇编程序将直接为初值表分配空间，无符号地址。设置变量名是为了方便存取它指示的存储单元。
- 初值表是用逗号分隔的参数，主要由数值常量，表达式或 ?、DUP组成。其中?表示初值不确定，即未赋初值；重复初值可以用DUP进行定义。DUP的格式为：

  重复次数 DUP（重复参数）

- 变量定义伪指令有DB/DW/DD/DF/DQ/DT，它们根据申请的主存空间单位分类，

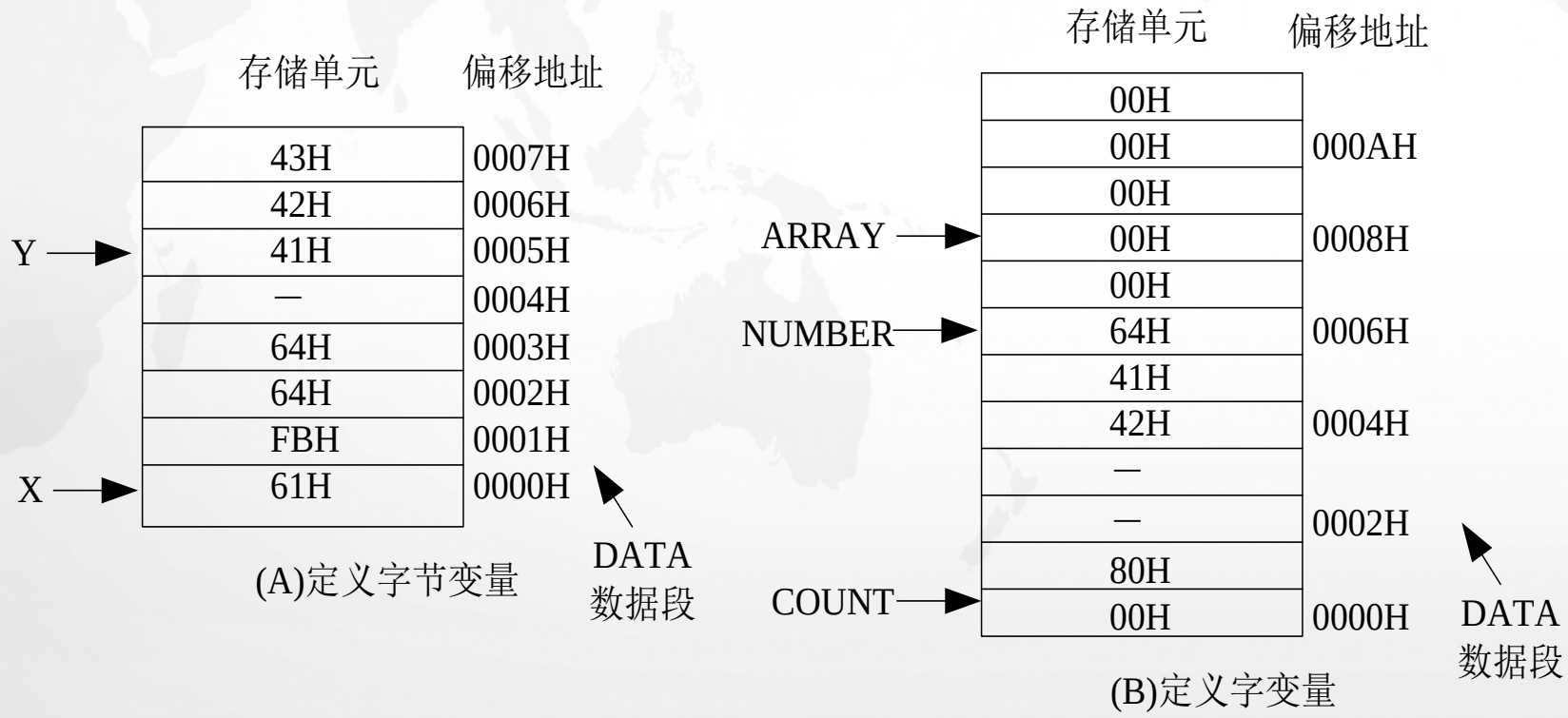
（1）定义字节单元伪指令DB

功能：定义变量的类型为BYTE，给变量分配字节或字节串。

要求：初值表中每个数据一定是字节量（Byte），可以是0～255的无符号数或是 −128～+127带符号数，也可以是字符串常量。例如：

```
DATA    SEGMENT                ；数据段
    X   DB          ‘a’，  −5
        DB          2       DUP（100），？
    Y   DB          ‘ABC’
DATA    ENDS
```

- 存储器中的分配情况如图


存储单元
偏移地址
43H
42H
41H
—
64H
64H
FBH
61H
0007H
0006H
0005H
0004H
0003H
0002H
0001H
0000H
Y
X
DATA
数据段
(A)定义字节变量
存储单元
偏移地址
00H
00H
00H
00H
00H
64H
41H
42H
—
—
80H
00H
000AH
0008H
0006H
0004H
0002H
0000H
ARRAY
NUMBER
COUNT
DATA
数据段
(B)定义字变量

数据定义的存储形式

针对上述的DATA定义，在执行下述汇编指令后，可以看到所定义的DATA中的数据发生了变化。

```
MOV     AL，X
        ；此处X表示它的第1个数据，故AL←‘a’
DEC     X+1
        ；对X为起始的第2个数据减1，故成为 −6
MOV  Y，AL          ；现在Y这个字符串成为‘ABC’
```

（2）定义字单元伪指令DW

功能：定义变量的类型为WORD，给变量分配一个或多个字单元，并可以将它们初始化为指定值。

要求：初值表中每个数据一定是字 （Word），一个字单元可用于存放任何16位数据，如一个段地址、一个偏移地址、两个字符、0～65535之间的无符号数或者是

–32768～+32767之间的带符号数。例如：

```
DATA        SEGM；数据段
    COUNT       DW          8000H，？，’AB’
   MAXINT       EQU   64H
   NUMBER       DW          MAXINT
   ARRAY        DW          MAXINT DUP（0）
DATA        ENDS
```

- 数据在存储器的分配情况如图所示

（3）定义双字单元伪指令DD

功能：定义变量的类型为DWORD，用于分配一个或多个双字单元，并将它们初始化为指定值。

要求：初值表中每个数据是一个32位的双字 （Double Word），可以是有符号或无符号的32位整数，也可以用来表达16位段地址（高位字）和16位的偏移地址（低位字）的远指针。

例如：

```
VARDD          DD     0，?，12345678H
FARPOINT   DD     00400078H
```

注意：在内存中存放时，低位字在前，高位字在后。

（4）其它数据定义伪指令

- 定义3字伪指令DF

功能：用于为一个或多个6字节变量分配空间及初始化。

用途：6字节常用在 32位CPU中表示一个48位远指针（16位段选择器：32位偏移地址）。

- 定义4字伪指令DQ

功能：用于为一个或多个8字节变量分配空间及初始化。

用途：8字节变量可以表达一个64位整数。

- 定义10字节伪指令DT

功能：用于为一个或多个10字节变量分配空间及初始化。

（5）定位伪指令

用数据定义伪指令分配的数据是按顺序一个接着一个存放在数据段中的。但有时，我们希望能够控制数据的偏移地址。例如使数据对齐可以加快数据的存取速度。MASM提供了这样的伪指令，这些伪指令我们称为定位伪指令。

ORG伪指令

伪指令格式：ORG参数

功能：ORG伪指令是将当前偏移地址指针指向参数表达的偏移地址。例如：

```
ORG 100h     ；从100h处安排数据或程序
ORG $+10          ；偏移地址加10，即跳过10个字节空间
```

汇编语言程序中，符号“$”表示当前偏移地址值。例如，在偏移地址100H单元开始定义“DW 1，2，$+4，$+4”，那么在104H单元的值为108H，106H单元的值为10AH。又如：

```
ARRAY          DB          12，34，56
LEN            EQU    $ – ARRAY
```

那么，LEN的值就是ARRAY变量所占的字节数。

EVEN伪指令

- 伪指令格式：EVEN
- 功能：EVEN伪指令使当前偏移地址指针指向偶数地址，即若原地址指针已指向偶地址，则不作调整；否则将地址指针加1，使地址指针偶数化。
- 用途：EVEN可以对齐字数据。

ALIGN伪指令

- 伪指令格式：ALIGN　　n
- 功能：ALIGN伪指令是将当前偏移地址指针指向n（n是2的乘方）的整数倍的地址，即若原地址指针已指向n的整数倍地址，则不作调整；否则将指针加以1～n–1中的一个数，使地址指针指向下一个n的整数倍地址。

例如：

```
DATA        SEGMENT      ；完整段定义
  DATA01              DB 1，2，3    ；DATA01的偏移地址为
    0000H
  EVEN                                    ；等价于ALIGN 2
  DATA02              DW 5          ；DATA02的偏移地址为
    0004H
  ALIGN 4
  DATA03              DD 6          ；DATA03的偏移地址为
    0008H
  ORG         $+10H
  DATA04              DB ‘ABC’      ；DATA04的偏移地址为
    001CH
DATA        ENDS
```

2变量和标号的属性

变量、标号、段名及过程名都表示的是地址，那么，这些标号和名字一经定义便具有以下三种属性：

- 段值：标号和名字对应存储单元所在段的段地址。
- 偏移值：标号和名字对应存储单元所在段的段内偏移地址。
- 类型：标号、子程序名的类型可以是NEAR（近）和FAR（远），分别表示段内或段间；变量名的类型可以是BYTE（字节），WORD（字）和DWORD（双字）等。

（1）地址操作符

地址操作符可获得名字或标号的段地址和偏移地址两个属性值。

- 中括号 [ ]表示将括起的表达式作为存储器地址指针；
- 符号 $ 表示当前偏移地址；
- 段前缀的冒号：也是一种地址操作符，它表示采用指定的段地址寄存器。
- 另外，还有两个经常应用的地址操作符OFFSET和SEG。

OFFSET操作符

- 使用格式：OFFSET名字/标号
- 功能：返回名字或标号的偏移地址

SEG操作符

- 使用格式：SEG名字/标号
- 功能：返回名字或标号的段地址

例如把字节变量ARRAY的段地址和偏移地址送入DS和BX就可用下列指令序列实现。

```
MOVAX，SEG ARRAY
MOVDS，AX
MOVBX，OFFSET ARRAY
                ；等价于LEA    BX，ARRAY
```

- 加、减运算符同样可以用于地址表达式，例如：

```
MOVCL，ARRAY+4
；等效于MOV CL，ARRAY[4]，这里的”4”表示4个字节单元
```

（2）类型操作符

- 类型操作符对名字或标号的类型属性进行有关设置。该类操作符有PTR、THIS、SHORT和TYPE。

①PTR操作符

- 使用格式：类型名 PTR 名字/标号
- 要求：PTR操作符中的“类型名”可以是BYTE/WORD/DWORD/FWORD/QWORD/TBYTE，或者是NEAR/FAR，还可以是由STRUCT，RECORD，UNION以及TYPEDEF定义的类型。
- 功能：使名字或标号具有指定的类型

例:

```
MOV    AL，BYTE PTR W_VAR
 ；W_VAR是一个字变量，BYTE PTR使其作为一个字节变量
JMP     FAR PTR N_LABEL
 ；N_LABEL是一个标号，FAR PTR使其作
；为段间转移使用PTR操作符，可以临时改变名字或标号的类
    型。
```

②THIS操作符

- 使用格式：THIS类型名
- 功能：创建当前地址，具有指定的类型。类型名同PTR操作符中的类型。例如：

```
B_VAR EQU THIS BYTE；按字节访问变量B_VAR，但与变量
                    ；W_VAR的地址相同
 W_VAR DW 10 DUP（0）  ；按字访问变量W_VAR
F _JUMP                EQU    THIS FAR
      ；用F_ JUMP为段间转移（F_JUMP LABEL  FAR）
N_JUMP：      MOV   AX，W_VSR
     ；用N_JUMP为段内近转移，但两者指向；同一条指令
```

MASM中还有一个LABEL伪指令，它的功能等同于“EQU THIS”。

③SHORT操作符

使用格式：SHORT标号

要求：转移范围为 −128～+127字节内。

功能：设定标号为短转移标号。SHORT指定标号作为

−128～+127字节范围内的短转移。例如：

JMP　　SHORT　　N_JUMP

④TYPE操作符

使用格式：TYPE　名字/标号

功能：返回一个属性为字数值，表明名字或标号的类型。

- 类型的返回数值

类型	返回数值
变量	该变量类型的每个数据占用的字节数
结构	每个结构元素占用的字节数
常量	0
标号	距离属性值
寄存器	该寄存器具有的字节数

- 操作符SIZEOF和LENGTHOF具有类似TYPE的功能，分别返回整个变量占用的字节数和整个变量的数据项数（即元素数）。实际上：
- SIZEOF返回值 = LENGTHOF返回值 × TYPE返回值
- 请注意，在MASM 5.x仅支持SIZE和LENGTH操作符。LENGTH对于变量定义使用 DUP的情况，返回分配给该变量的单元数，其它情况为1。SIZE返回LENGTH与TYPE的乘积。

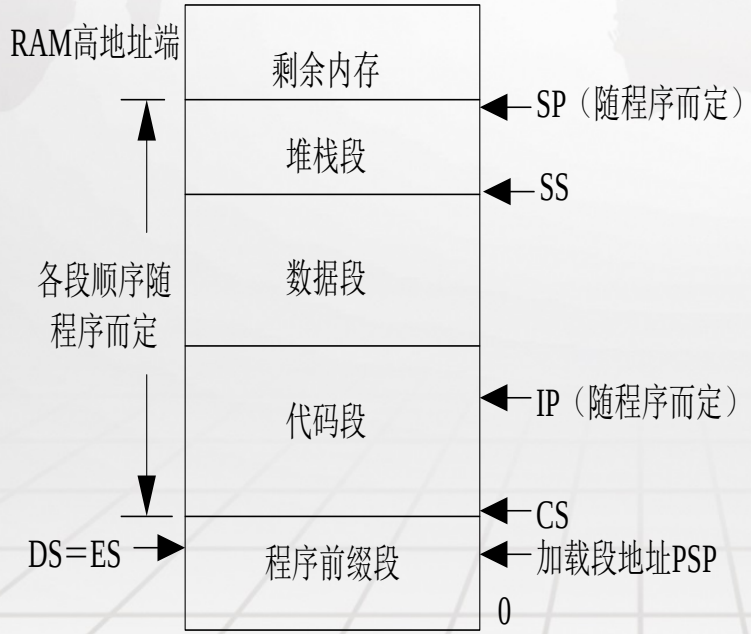


程序段的定义和属性

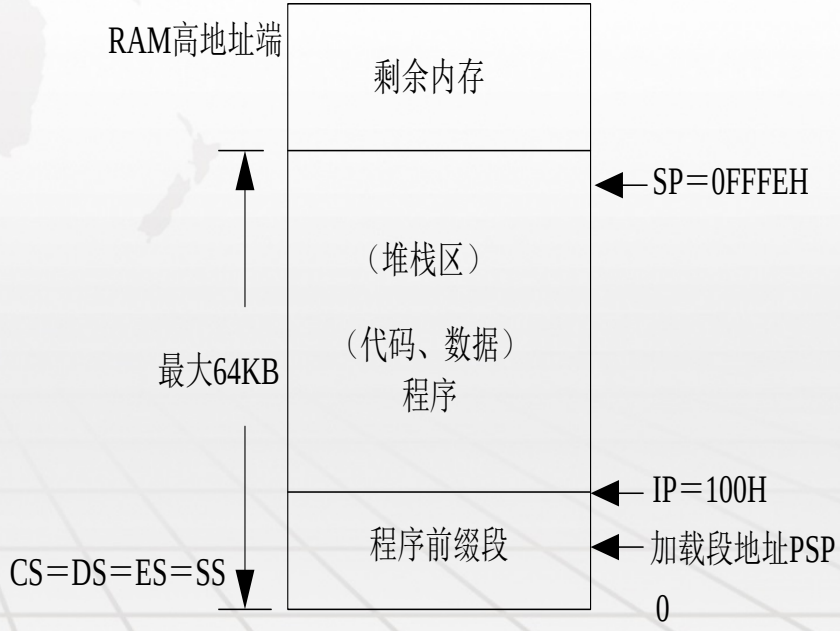
PART 03

DOS的程序结构

1. EXE程序



2. COM程序



简化段定义的格式

```
.MODEL      SMALL ; 定义程序的存储模式（一般采用
  SMALL）
.STACK              ; 定义堆栈段
.DATA               ; 定义数据段
      . . .         ; 数据定义
.CODE         ; 定义代码段
.STARTUP      ; 程序起始点，并建立DS，SS内容（注1）
      . . .         ; 程序代码
.EXIT 0             ; 程序结束点，返回DOS（注2）
      . .           ; 子程序代码
END                 ; 汇编结束（注3）
```

1.存储模式伪指令

伪指令格式：MODEL存储模式[，语言类型][，操作系统类型][，堆栈选项]

MASM有7种不同的存储模式

TINY（微型模式）

SMALL（小型模式）

COMPACT（紧凑模式）

MEDIUM（中型模式）

LARGE（大型模式）

HUGE（巨型模式）

FLAT（平展模式）

2. 简化段定义伪指令

（1） .STACK堆栈段伪指令

（2） .DATA数据段伪指令

（3） .CODE 代码段伪指令

3. 程序开始伪指令

```
.STARTUP
```

4. 程序终止伪指令

```
.EXIT[返回数码]
```

```
.EXIT 0 等效于
MOV       AX, 4C00H
INT       21H
```

5. 汇编结束伪指令 END

完整段定义的格式

```
STACK   SEGMENT                    ; 定义堆栈段STACK
                 . . .                      ; 分配堆栈段的大小
STACK           ENDS               ; 堆栈段结束
DATA            SEGMENT            ; 定义数据段DATA
                 . . .             ; 定义数据
DATA            ENDS                       ; 数据段结束
 CODE   SEGMENT  'CODE'            ; 定义代码段CODE
ASSUME CS: CODE, DS: DATA, SS: STACK     ; 确定CS/DS/SS指向
                                   ;的逻辑段
   START:       MOV     AX, DATA           ; 设置数据段的段地址DS
                        MOV     DS, AX
                . . .                      ; 程序代码
                        MOV     AX, 4C00H          ; 返回DOS
                        INT     21H
                . . .              ; 子程序代码
   CODE                 ENDS               ; 代码段结束
END   START                        ; 汇编结束，程序起始位置为START
```

例 设计一个COM程序，实现按任意键后响铃的功能。

```
  .MODEL      TINY                          ；采用微型模式
  .CODE                                     ；只有一个段，没有数据段和堆栈段
  .STARTUP                                  ；等效于ORG 100H，汇编程序自动产生
    MOV       DX，OFFSET STRING             ；显示信息
    MOV       AH，9
    INT       21H
    MOV       AH，01H                       ；等待按键
    INT       21H
    MOV       AH，02H                       ；响铃
    MOV       DL，07H
    INT       21H
  .EXIT 0
STRING DB ‘PRESS ANY KEY TO CONTINUE !$‘
    ；数据安排在不与代码冲突的地方
END
```

对于一个典型的MASM程序，完整段可用下述格式定义。

```
STACK        SEGMENT STACK      ；定义堆栈段STACK
                 . . .          ；分配堆栈段的大小
STACK        ENDS               ；堆栈段结束
DATA         SEGMENT    ；定义数据段DATA
                 . . .          ；定义数据
DATA         ENDS               ；数据段结束
CODE         SEGMENT    ‘CODE’  ；定义代码段
  CODE
ASSUME CS：CODE，DS：DATA，SS：STACK
          ；确定CS/DS/SS指向的逻辑段
```

```
START:      MOV   AX，DATA      ；设置数据段的段地址DS
            MOV   DS，AX
．．．                ；程序代码
            MOV   AX，4C00H     ；返回DOS
            INT         21H
                  ．．．                          ；子程序代码
  CODE            ENDS                            ；代码段结束
END  START        ；汇编结束，程序起始位置为START
```

1．完整段定义伪指令

- 完整段定义由SEGMENT和ENDS这一对伪指令实现。
- 伪指令格式：
- 段名　　SEGMENT［定位］［组合］［段字］［‘类别’］
  　　．．．　　；语句序列
- 段名　　ENDS
- 功能：SEGMENT伪指令定义一个逻辑段的开始，ENDS伪指令表示一个段的结束。

(1).段定位（Align）属性

指定逻辑段在主存储器中的边界，该关键字可表示为：

- BYTE：段开始地址为字节地址（xxxx xxxxB），属性值为1；
- WORD：段开始地址为字地址 （xxxx xxx0B），属性值为2；
- DWORD:段开始地址为4的倍数 （xxxx xx00B），属性值为4
- PARA：段开始地址为16的倍数（xxxx 0000B），即节地址，属性值为16
- PAGE：表示段开始地址256的倍数（0000 0000B），即页地址，属性值为256

(2) 段组合（Combine）属性

- 指定多个逻辑段之间的关系。组合的关键字有：
  - PRIVATE：
  - PUBLIC：
  - STACK：
  - COMMON：

(3) 段字（Use）属性

- 这是为支持32位段而设置的属性。

(4) 段类别（Class）属性

- 当连接程序组织段时，将所有的同类别段相邻分配。
- 段类别可以是任意名称，但必须位于单引号中；
- 大多数MASM程序使用‘CODE’，‘DATA’和‘STACK’来分别指名代码段、数据段和堆栈段，以保持所有代码和数据的连续。

2. 指定段寄存器伪指令

- 伪指令格式：ASSUME　段寄存器：段名[，段寄存器名：段名，．．．]
- 功能：ASSUME伪指令通知MASM用指定的段寄存器来寻址对应的逻辑段，即建立段寄存器与段的缺省关系。

```
CODE   SEGMENT
ASSUME CS：CODE，DS：DATA，SS：STACK，ES：DATA
                  MOV     AX，DATA
                  MOV     DS，AX
                  MOV     ES，AX
                  MOV     AX，STACK
                  MOV     SS，AX
                  ┇
CODE   ENDS
```

3．GROUP段组伪指令

- 伪指令格式： 组名 GROUP 段名[，段名，．．．]
- 功能：伪指令GROUP把多个同类段合并为一个64KB物理段，并用一个组名统一存取它。
- 例如：

```
DATA1 SEGMENT    WORD PUBLIC  ‘CONST’；常量数据段
    CONST1              DW          100
DATA1               ENDS
DATA2 SEGMENT WORD PUBLIC ’ VARS’  ；变量数据段
    VAR1            DW  ?
DATA2               ENDS
DATAGROUP GROUP DATA1, DATA2; 进行组合
```

4．段顺序伪指令

段在主存中的实际顺序是可以设置的.

(1)SEG段顺序伪指令

- 伪指令格式： .SEG
- 功能：按照源程序的各段顺序排序。

(2)DOSSEG段顺序伪指令

- 伪指令格式：.DOSSEG
- 功能：其它微软的程序设计语言按照标准DOS规定进行排序，即代码段、数据段、堆栈段。

(3)ALPHA段顺序伪指令

- 伪指令格式： .ALPHA
- 功能：按照段名的字母顺序排序。

5. 定位伪指令

- **（1）ORG伪指令**

```
ORG 100h          ；从100h处安排数据或程序
ORG $+10     ；使偏移地址加10，即跳过10个字节空间
```

- **（2）EVEN伪指令**
  - EVEN伪指令使当前偏移地址指针指向偶数地址，即若原地址指针已指向偶地址，则不作调整；否则将地址指针加1，使地址指针偶数化。
  - 用途：EVEN可以对齐字数据。

- **（3）ALIGN伪指令**
  - 将当前偏移地址指针指向n（n是2的乘方）的整数倍的地址，即若原地址指针已指向n的整数倍地址，则不作调整；



复杂数据结构

PART 04

结构

1．结构类型的说明

伪指令格式：

结构名　　STRUCT

　　．．．　　；数据定义语句

结构名　　ENDS

例如：下述语句段说明了学生成绩结构。

```
STUDENT STRUCT
        SID     DW      ?
        SNAME   DB      'ABCDEFGH'
        MATH    DB      0
        ENGLISH DB      0
STUDENT ENDS
```

2．结构变量的定义

- 变量名　结构名　<字段初值表>

STU1　　STUDENT　　<1，’ZHANG’，85，90>

STU2　　STUDENT　　<2，’WANG’，，>

STUDENT　　100 DUP（< >）

；预留100个结构变量空间

3．结构变量及其字段的引用

- 结构变量名.结构字段名
- MOV　　STU1.MATH，95
  ；执行指令后，将对MATH域
  ；的值更新为95。

记录

1．记录类型的说明

记录名RECORD位段[，位段．．．]

记录中位段的格式定义如下：

位段名：位数[＝表达式]

例如：

PERSON　RECORD　YEAR：4，　SEX：1=0，　MARRIAGE：1=1

2．记录变量的定义

记录变量定义的格式为：

记录变量名　记录名　<段初值表>

例如：

```
ZHANG PERSON  <1000B, 1, 0>    ；该字节值为：00100010B=22H
 WANG         PERSON          <1001B, 0, 0>       ；该字节值为：
   00100100B= 24H
```

3．记录变量的引用和记录操作符

- 记录变量通过它的变量名直接引用，表示它的字节或字值



宏汇编

PART 05

- 宏是具有宏名的一段汇编语句序列。
- 经过定义的宏，只要写出宏名，就可以在源程序中调用它。

宏的定义和调用

1．宏定义

宏定义由一对宏汇编伪指令MACRO/ENDM来完成。格式：

```
宏名   MACRO[形参表]
       宏定义体
ENDM
```

例如：

```
MAINBEGIN   MACRO ；定义名为MAINBEGIN的宏，无参数
   MOVAX, @DATA   ；宏定义体
  MOV DS, AX
ENDM                ；宏定义结束
```

2. 宏调用

宏定义之后就可以使用它，即宏调用。宏调用遵循先定义后调用的原则。宏调用格式为：

伪指令格式：宏名[实参表]

（1）宏定义中可以有宏调用，只要遵循先定义后调用的原则。

例如：

```
DOSINT21    MACRO    FUNCTION          ；宏定义
            MOV      AH, FUNCTION
            INT      21H
            ENDM
DISPMSG MACRO MESSAGE      ；含有宏调用的宏定义
            MOV   DX, OFFSET MESSAGE
            DOSINT21 9                                    ；
宏调用
            ENDM
```

上述宏定义汇编后的列表文件如下。

```
    DISPMSG MSG                    ; 宏调用
    MOV     DX, OFFSET MSG         ; 宏展开（第一层）
    DOSINT21      9
    MOV     AH, 9                  ; 宏展开（第二层）
    INT     21H
```

（2） 宏定义允许嵌套

即宏定义体内可以有宏定义，对这样的宏进行调用时需要多次分层展开。宏定义内也允许递归调用，这种情况需要用到后面将介绍的条件汇编指令给出递归出口条件。

宏的参数

例 具有多个参数的宏定义。

使用8086的移位指令有时感到不便，因为当移位次数大于1时，必须利用CL寄存器。现在用宏指令SHLEXT扩展逻辑左移SHL的功能。例如：

```
SHLEXT    MACRO     SHLOPRAND, SHLNUM
      PUSH      CX
      MOV CL, SHLNUM    ; SHLNUM表示移位次数
      SHL SHLOPRAND, CL ; SHLOPRAND表示被
                              ; 移位的操作数
      POP CX
            ENDM
```

当我们要将AX左移6位时，可以采用如下宏指令：

```
SHLEXT          AX, 6
```

汇编后，宏展开为：

```
PUSH        CX
MOV CL, 06
SHL AX, CX
POP CX
```

宏操作符：

&　　替换操作符，用于将参数与其它字符分开。

< >　　字符串传递操作符。用于括起字符串。

!　　转义操作符。用于指示其后的一个字符作为一般字符，而不含特殊意义。

%　　表达式操作符。用在宏调用中，表示将随后的一个表达式的值作为实参，而不是将表达式本身作为参数。例如，已经有宏定义DSTRING（见下一页），那么：

```
DSTRING    %（1024 - 1）      ；宏调用
DB  ‘1023’，0DH，0AH，’$’          ；宏展开
```

;　　宏注释符。用于表示在宏定义中的注释。采用这个符号的注释，在宏展开时不出现。

另外，宏定义中还可以用“：REQ”说明设定不可缺参数，用“：＝默认值”设定参数默认值。

例　用于字符串的宏定义参数

```
DSTRING     MACRO  STRING
    DB        '& STRING&', ODH, OAH, '$'
            ENDM
```

例如，要定义字符串‘THIS　IS　A　EXAMPLE.’，可以采用如下宏调用。

```
DSTRING    <THIS  IS  A  EXAMPLE.>
```

它产生的宏展开为：

```
 DB   ‘THIS  IS  A  EXAMPLE.’, ODH,
OAH,’ $’
```

与宏有关的伪指令

1．局部标号伪指令LOCAL

问题提出：　宏定义可被多次调用，当宏定义中使用了标号，同一源程序对它的多次调用就会造成标号的重复定义，汇编将出现语法错误。

解决：如果宏定义体采用了标号，可以使用局部标号伪指令LOCAL加以说明。

伪指令格式为：LOCAL　标号列表

每次宏展开时汇编程序将对其中的标号自动产生一个唯一的标识符（其形式为“??0000”到“??FFFF”），避免宏展开后的标号重复

注意：LOCAL伪指令只能在宏定义体内使用，而且是宏定义MACRO语句之后的第一条语句，两者间也不允许有注释和分号。

```
ABSOL MACRO OPRD
      LOCAL NEXT
      CMP   OPRD, 0
      JGE   NEXT
      NEG   OPRD
NEXT:
      ENDM ；这个伪指
        ；令要独占一行
```

采用例4.15宏定义的宏调用形式为：

```
    ABSOL   WORD PTR [BX]
    ABSOL   BX
```

上述宏调用下的宏展开为：

```
        CMP WORD PTR [BX], 0
        JGE ??0000
        NEG WORD PTR [BX]
?? 0000:
        CMP BX, 0
        JGE ?? 0001
        NEG BX
?? 0001:
```

2．宏定义删除伪指令PURGE

当我们不需要某个宏定义时，可以把它删除，删除宏定义伪指令的格式为：

PURGE　　宏名表

允许宏名与其它指令包括伪指令同名，此时宏名优先级最高。当不再使用这个宏定义时，及时用PURGE删除即恢复原指令功能。

3. 宏定义退出伪指令EXITM

- EXITM
- 它可用于宏定义体、重复汇编的重复块以及条件汇编的分支代码序列中，汇编程序执行EXITM指令后立即停止它后面部分的宏展开。

宏与子程序

- 共同点
  - 都可以把一段程序用一个名字定义，简化源程序的结构和设计。
- 区别：
  - 宏调用在汇编时进行程序语句的展开，不需要返回；子程序调用在执行时由CALL指令转向子程序体，需要执行RET指令返回；
  - 宏调用仅是源程序级的简化，并不减小目标程序，子程序还是目标程序级的简化，形成的目标代码较短

- 宏调用简单、直接，不需要返回；子程序需要利用堆栈保存和恢复转移地址、寄存器等，要占用一定的时空开销；
- 宏调用的参数通过形参、实参结合实现传递，子程序需要利用寄存器、存储单元或堆栈等传递参数。
- 对宏调用来说，参数传递错误通常是语法错误，会由汇编程序发现；而对子程序来说，参数传递错误通常反映为逻辑或运行错误，不易排除。
- 当程序段较短或要求较快执行时，应选用宏；当程序段较长或为减小目标代码时，要选用子程序。



常用的系统功能调用

PART 06

- DOS系统是单用户的操作系统DOS系统提供给用户的编程界面大体有如下几种：
  - 裸机层软件开发，利用芯片或板卡（适配器）支持的寄存器或内存数据区编程。需要用户非常清楚设备的硬件细节，编程较复杂，但软件实现速度最快。
  - BIOS级软件开发，利用BIOS基本输入输出系统所提供的一些服务功能编程。
  - 系统功能级软件开发，利用DOS系统提供的系统功能编程。

BIOS功能级软件接口及实现

1．BIOS级软件接口概述

- BIOS基本输入输出系统由一批子程序组成，负责管理系统内的输入输出设备
- 直接为DOS操作系统和应用程序提供底层设备驱动服务。
- 大多数的驱动程序以软件中断方式调用，少数由硬件中断调用。

常用BIOS服务功能

BIOS服务	功能号	功　　能
打印屏幕服务	05H	将当前视频页内容送到默认打印机
视频服务	10H	为显示适配器提供I/O支持
软盘服务	13H	提供软盘的读、写、格式化、初始化、诊断
硬盘服务	13H	提供硬盘的读、写、格式化、初始化、诊断
串行通信服务	14H	为串行适配器提供字符输入/输出
系统服务	15H	系统级子服务
键盘服务	16H	为键盘提供I/O支持
并行打印机服务	17H	为并行打印机提供I/O支持
日期时间服务	1AH	设置和读取时间、日期、声源等

2．视频服务

- INT　10H
  - AH寄存器选择视频服务功能
  - 待写的字符或像素值一般在AL寄存器中传递。
  - 功能调用保存BX、CX、DX及段寄存器的值。其他寄存器的内容（特别是SI、DI）不保存
  - *X*坐标（列号）在CX（图形功能）中或DL（正文功能）中传递
  - 显示页在BH中传递，显示页从零开始计数

例如：利用BIOS视频服务的AH=0CH子功能实现写像素点。

- 调用子程序之前，需要用DX保存行号，CX保存列号，AL保存颜色值，这是0CH子功能要求的。

```
WRITINGPIXEL        PROC            NEAR
        PUSH        AX
        MOV         AH, 0CH
        INT         10H
        POP         AX
WRITINGPIXEL                ENDP
```

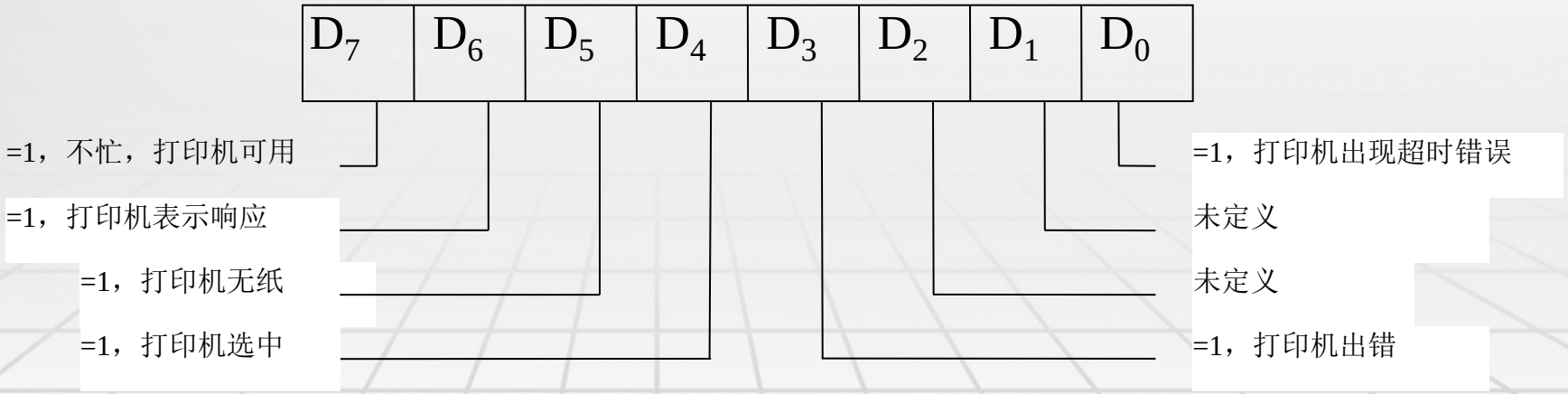
3．键盘服务

- 键盘ISR（键盘中断服务程序） INT 09H硬件中断
- 键盘DSR（键盘设备服务程序），INT 16H软件中断
  - BIOS键盘服务包括3个子功能：0号功能、1号功能、2号功能

子功能号	含　　义	出　口　参　数
AH=0	从键盘输入一个字符	AL=ASCII码（或0） AH=扫描码（或扩展扫描码）
AH=1	判断键盘有无字符输	ZF=0 有键按下，键代码保存在AX中 ZF=1 无键按下
AH=2	当前键盘特殊键状态	AL=KB-FLAG的变量

4. 并行打印机服务

- INT 17H
  - 0号功能（给打印机传送一个字符）
  - 1号功能（初始化打印机）
  - 2号功能（读打印机状态）
- 均可通过AH返回打印机状态字节



DOS功能级软件接口及调用实例

1．DOS功能级软件接口调用概述

- **主要包括：磁盘管理、内存管理、基本输入输出管理等。**
- 子功能都有一个00H~57H的功能编号
- INT 21H

- 00H~0CH ：字符I/O管理。包括键盘、显示器、打印机、异步通信口的管理
- 0DH~24H：文件管理。包括复位、选择磁盘，打开、关闭、删除文件，顺序读、写文件，建立文件，重命名文件，查找驱动器分配表信息，随机读、写文件，查看文件长度
- 25H~26H：非设备系统调用。包括设置中断向量，建立新程序段
- 27H~29H：文件管理。包括随机块读写、分析文件名
- 2AH~2EH：非设备系统调用。包括读取、设置日期、时间
- 2FH~38H：扩充的系统调用。包括读取DOS版本号，终止进程，读取中断矢量，读取磁盘空闲空间
- 39H~3BH：目录组。包括建立子目录，修改当前目录，删除目录项

- 3CH~46H：扩充的文件管理。包括建立、打开、关闭文件，从文件或设备读写数据，在指定路径删除、移动文件，修改文件属性，设备I/O控制，复制文件标志47H取当前目录组。
- 48H~4BH：扩充的内存管理。包括分配内存、释放已分配的内存、分配内存块、装入或执行程序等
- 4CH~4FH：扩充的系统调用。包括终止进程、查询子程序的返回代码、查找第一个相匹配的文件、查找下一个相匹配的文件
- 50H~53H：扩充的系统调用。供DOS内部使用
- 54H~57H：扩充的系统调用。包括读取校验状态、重新命名文件、设置读取日期及时间

2. 基本I/O功能调用

- 键盘输入（1号调用）
- 无回显的键盘输入（8号调用）
- 控制台输入输出（6号调用）
- 无回显的控制台输入（7号调用）
- 打印输出（5号调用）
- 输出字符串（9号调用）
- 输入字符串（0AH调用）
- 异步通信输入输出（3、4号调用）
- 日期与时间的设置与获取（2AH、2BH、2CH、2DH调用）

3. DOS功能调用实例

```
        ;利用9号功能显示字符串
        MOV         AX, MYDATA
        MOV         DS, AX
DISP:   MOV         DX, OFFSET MESSEGE
        MOV         AH, 09H
        INT         21H
        ; 利用10号功能接收字符到缓冲区
KEY:    MOV         DX, OFFSET PARAMETERS
        MOV         AH, 10
        INT         21H
```



顺序结构程序设计

PART 07

- 汇编语言源程序主体（代码段）可以有顺序、分支、循环、子程序和宏等结构。
- 早期版本的汇编程序不直接支持结构化程序设计，可以用微处理器指令系统中的转移指令、循环指令、子程序调用及返回指令，实现程序的各种结构形式。

- **例** 求两个数的平均值。这两个数分别放在X单元和Y单元中，而平均值放在Z单元中。

根据题意，所设计的程序如下：

```
.MODEL   SMALL
.STACK
.DATA
X        DB     8CH
Y                DB     64H
Z        DB     ?
```

```
.CODE
.STARTUP
        MOV     AL, X           ; AL←8CH
        ADD     AL, Y           ; AL←8CH+64H
        MOV     AH, 00H         ; AH←00H
        ADC     AH, 00H         ; 进位送AH
        MOV     BL, 2           ; 除数2→BL
        DIV     BL              ; AX除以BL的内容,
                                ; 商→AL, 余数→AH
        MOV     Z, AL           ; 结果送入Z单元
                .EXIT 0
END
```

7.1.1-汇编语言基本程序设计——顺序程序_哔哩哔哩_bilibili



分支程序设计

PART 08

无条件转移指令和条件转移指令

1. 无条件转移指令

在8086/8088系统中，程序的寻址由CS和IP两部分组成。改变CS和IP或只改变IP都能使程序转移到一个新的地址去执行。

指令格式：JMP　dest

JMP　reg/m16

指令功能：JMP指令无条件地转移到指令所指定的目标地址，dest标号提供转移目的地址，或者由寄存器、存储器提供转移目的地址。

根据目标地址相对于转移指令的位置，转移可分为短转移、段内转移和段间转移。

- 短（SHORT）转移属于相对转移，指在段内短距离（-128～127）转移。
- 段内（NEAR）转移指CS值不变，只给出地址偏移值的转移；这种转移的目标地址与转移指令都在同一段内。
- 段间（FAR）转移指CS段值和IP值都发生改变的转移。在这种情况下，程序从一个段转移到另一段中的某一地址去执行，因此，JMP指令中要同时给出段值和偏移值。

例

```
JMP  NEAR PTR TABLE[BX]     ; 为段内转移,
   IP←[BX+TABLE][BX+TABLE +1]
JMP  FAR PTR TABLE[BX]      ; 为段间转移,
   IP←[BX+TABLE][BX+TABLE +1]
;          CS←[BX+TABLE+2][BX+TABLE +3]
```

在16位寻址方式下，可以用WORD和DWORD来区分不带标号的转移是段内转移还是段间转移。

例 3.60

```
JMP  WORD PTR [BX]          ; 为段内转移,
   IP←[BX][BX+1]
JMP  DWORD PTR [BX]         ; 为段间转移,
   IP←[BX][BX+1], CS←[BX+2][BX+3]
```

2. 条件转移指令

1）无符号数条件转移指令格式及其功能

JA/JNBE　　label；高于/不低于等于，条件满足则转移到label指定的地址。

JAE/JNB　　label；高于等于/不低于，条件满足则转移到label指定的地址。

JB/JNAE　　label；低于/不高于等于，条件满足则转移到label指定的地址。

JBE/JNA　　label；低于等于/不高于，条件满足则转移到label指定的地址。

JC　　label ；有进位，条件满足则转移到label指定的地址。

JE/JZ label ；等于/为0，条件满足则转移到label指定的地址

JNC　　label ；无进位，条件满足则转移到label指定的地址。

JNE/JNZ label　　；不等于/不为0，条件满足则转移到label指定的地址

JNP/JPOlabel ；非偶/奇，条件满足则转移到label指定的地址

JP/JPE　　label　　；偶/偶，条件满足则转移到label指定的地址。

2）带符号数条件转移指令格式及其功能

- JG/JNLE　label　　；大于/不小于等于，条件满足则转移到label指定的地址。
- JGE/JNL　label　　；大于等于/不小于，条件满足则转移到label指定的地址。
- JL/JNGE　label　　；小于/不大于等于，条件满足则转移到label指定的地址。
- JLE/JNG　label　　；小于等于/不大于，条件满足则转移到label指定的地址。
- JNO　　label　　；无溢出，条件满足则转移到label指定的地址。
- JNS　　label　　；正数或0，条件满足则转移到label指定的地址。
- JO　label；溢出，条件满足则转移到label指定的地址。
- JS　label；负数，条件满足则转移到label指定的地址。

3）据CX中的值来决定转移的指令格式及其功能

- JCXZ　　label　；CX中的值为0，则转移到label指定的地址。

分支结构程序设计举例

- 单分支IF - THEN
- 双分支IF - THEN—ELSE
- 条件转移Jcc和无条件转移JMP指令用于实现程序的分支结构。
  - JMP指令仅实现了转移到指定位置，
  - Jcc指令则可根据条件转移到指定位置或不转移而顺序执行后续指令序列。

1．单分支结构

例 计算AX中符号数绝对值的程序段。

```
        CMP     AX，0
        JGE     NONNEG   ；分支条件：AX≥0
        NEG     AX       ；条件不满足，负数，
                         ;执行分支体进行求补
NONNEG：MOV   RESULT，AX ；条件满足，
                         ;为正数，保存结果
```

```
        CMP    AX, 0
        JL     YESNEG          ; 分支条件: AX < 0
        JMP     NONENG         ; 条件不满足，正数，
                               ;转向保存结果
YESNEG: NEG AX                 ; 条件满足，为负数，
                               ;需要求补
NONNEG: MOV RESULT, AX   ; 保存结果
```

选择分支条件不当，不仅多了一个JMP指令，而且也容易出错。

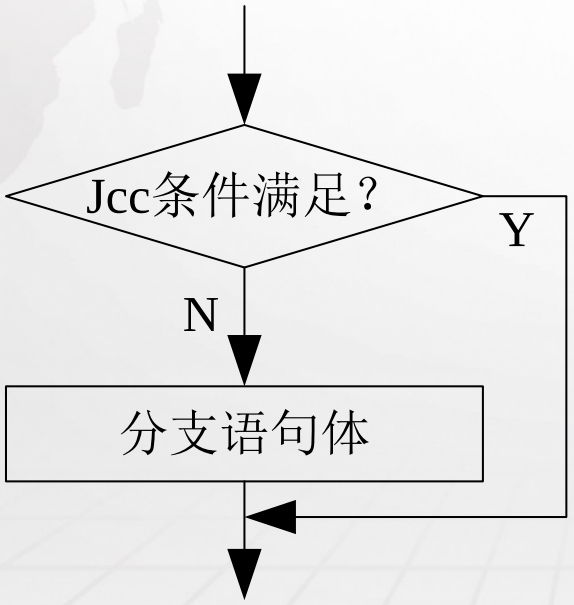
2．双分支结构

- 对于双分支程序，顺序执行的分支语句体1不会自动跳过分支语句体2，所以分支语句体1最后一定要有一条JMP指令跳过分支体2，即分支汇点处；否则将进入顺序分支语句体2而出现错误 。

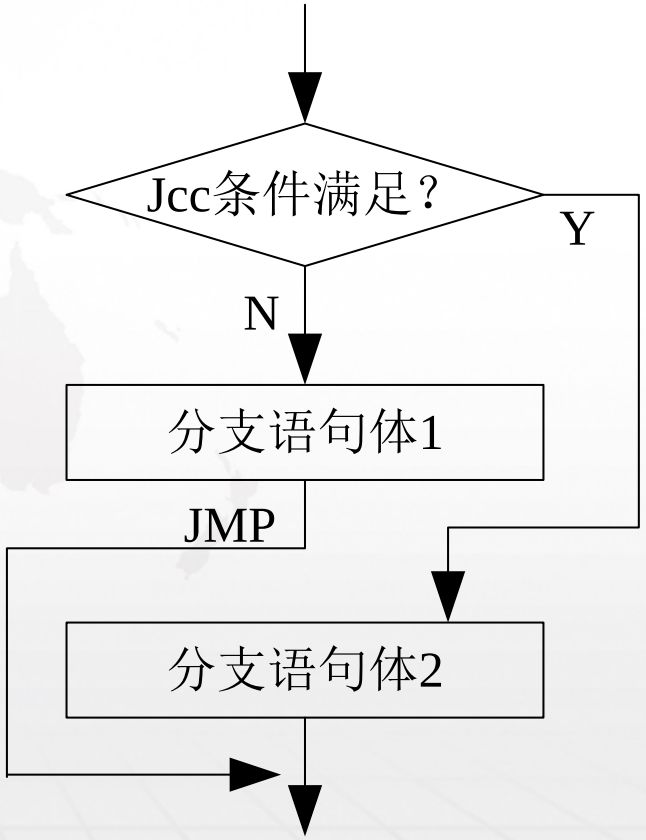
例 显示BX最高位的程序段。

```
      SHL    BX, 1     ; BX最高位移入CF标志
      JC     ONE       ; CF＝1，即最高位为1，转移
      MOV    DL, ' 0'      ; CF＝0，即最高位为0
                       ; DL←' 0'
      JMP    TWO       ; 一定要跳过另一个分支体
ONE:  MOV    DL, ' 1'      ; DL← '1'
TWO:  MOV    AH, 2
      INT    21H       ; 显示
```

- 单分支与双分支



A）单分支结构



B）双分支结构

3．分支程序设计

7.2.1-汇编语言基本程序设计——分支程序_哔哩哔哩_bilibili



循环程序设计

PART 09

循环指令

循环指令实际上也令LOOP是条件转移指令，它用存放在CX中的数作为循环重复计数值递减计数，直到CX中的数为0时终止循环。

1）LOOP循环指令

指令格式： LOOP label

指令功能：在执行LOOP指令时，处理器先将CX中计数值减1，再判其值：如果计数值不为0，则转到LOOP指令中指出的目标语句（执行循环体内的语句）；如果计数值为0，则执行LOOP指令后的下条指令（退出循环）。如果CX的初值为0，则循环将进行2^{16}次。

2）LOOPE/LOOPZ循环指令

指令格式：LOOPE　label

LOOPZ　label

LOOPE和LOOPZ是同一条指令的两种不同的助记符，其指令功能是：

执行本指令时，将CX寄存器的内容减1并回送CX寄存器，ZF不受CX减1的影响。

这两条指令执行后是否循环，依下述条件判断：

如果CX≠0且ZF=1，则转移到目的标号执行，IP←IP（现行）+偏移量。

如果CX≠0且ZF=0，则停止循环，按指令顺序执行。

如果CX=0（无论ZF如何）停止循环，按指令顺序执行。

3）LOOPNE/LOOPNZ循环指令

指令格式：LOOPNE/ LOOPNZ label

LOOPNE和LOOPNZ是同一条指令的两种不同的助记符，其指令功能：

将CX寄存器的内容减1并回送CX寄存器，ZF不受CX减1的影响。

- 判断是否循环的条件是：

如果CX≠0且ZF=0，继续循环，转到目的标号执行，即IP←IP（现行）+偏移量。

如果CX≠0且ZF=1，则停止循环，按指令顺序执行。

如果CX=0停止循环，按指令顺序执行。

注意：LOOPE/LOOPZ与LOOPNE/LOOPNZ指令对标志位没有影响

循环程序设计举例

1．循环结构

- **循环初始部分**　　开始循环准备必要的条件，如循环次数、循环体需要的初始值等。
- **循环体部分**　是循环工作的主要部分，是为完成某种特定功能而设计的重复执行的程序段。
- **修改部分**　　对循环条件、相关信息（如计数器的值、操作数地址等）进行修改的程序段。
- **循环控制部分**　　判断循环条件是否成立，决定是否继续循环，

2．循环程序设计

7.3.1-汇编语言基本程序设计——循环程序_哔哩哔哩_bilibili



串处理程序设计

PART 10

8086串操作指令

- 一般情况下，串操作处理的数据指在内存中地址连续的字节串（或字串），也就是处理一组数据或字符，数据串的长度最长为64KB。
- 串操作类指令要求SI为源串操作数指针，DI为目标操作数指针，每条指令一般只处理一个字节（或字）数据，而且每处理完一个数据，其地址指针会自动±1（或±2），指向下一个要处理的数据。指针是增还是减取决于状态寄存器（FLAGS）中DF的状态（DF=0，则增址；DF=1，则减址）。指针是增（减）1还是增（减）2取决于字节操作还是字操作。
- 单独的串操作指令只能对单个串元素进行操作，只有在这些指令前加上重复前缀REP，才能实现对整个串的操作；在重复前缀的作用下，串指令的基本操作不断重复。

(1) 串传送指令

指令格式：MOVSB、MOVSW

指令功能：串传送指令MOVSB、MOVSW分别将DS：SI所指向的串元素移送到由ES：DI指向的位置。MOVSB作用于字节元素，MOVSW作用于字元素，根据DF的值确定增值或减值SI与DI。

当串传送指令与重复前缀REP一起使用时，才能完成将整个串从源存储区域传送到目的存储区域的操作。为此，程序必须在执行串传送指令前设置好CX、SI和DI的值。

(2) 串比较指令

指令格式：CMPSB、CMPSW

指令功能：串字节比较指令CMPSB把寄存器SI所指向的一个字节数据与由寄存器DI所指向一个字节数据采用相减方式比较，相减结果影响有关标志位(AF，CF，OF，PF，SF和ZF)，但不会影响两个操作数，然后根据方向标志位DF的使SI和DI之值分别增1或减1。串字比较指令CMPSW把寄存器SI所指向的一个字数据与由寄存器DI所指向的一个字数据比较，结果影响有关标志位，但不会影响两个操作数，然后根据方向标志位DF的使SI和DI之值分别增2或减2。

(3) 串扫描指令

指令格式：SCASB、SCASW

指令功能：串字节扫描指令SCASB把累加器AL的内容与由寄存器DI所指向一个字节数据采用相减方式比较，相减结果影响有关标志位(AF，CF，OF，PF，SF和ZF)，但不影响两个操作数，然后根据方向标志DF使之值增1或减1。串字扫描指令SCASW把累加器AX的内容与由寄存器DI所指向的一个字数据比较，结果影响标志，然后根据方向标志DF使DI之值增2或减2。

（4）串装入指令

指令格式：LODSB

LODSW

指令功能：串装入指令是将寄存器SI所指的字节或字数据装入AL或AX，如果操作数的类型为字节，则采用LODSB指令；如果操作数的类型为字，则采用LODSW指令。使用上述格式的串装入指令时，仍必须先给SI赋合适的值。

(5) 串存储指令

指令格式：STOSB

STOSW

指令功能：字符串存储指令只是把累加器的值存到字符串中，即替换字符串中的一个字符。串存储指令STOSB/STOSW把累加器AL/AX的内容送到寄存器DI所指向的存储单元中，然后根据方向标志DF使DI值增1/2或减1/2。

(6) 重复前缀

指令格式：REP

REPE/REPZ

REPNE/REPNZ

这三种重复前缀指令不能单独使用，只能加在串操作指令之前，用来控制跟在其后的字符串操作指令，使之重复执行，重复前缀不影响标志位。

1） REP

无条件重复前缀。

指令功能：该指令无条件执行其后的指令，由CX寄存器指定重复次数，每执行一次，则CX←CX-1，若CX≠0，继续执行其后的指令，一直到CX=0为止。REP前缀常与MOVS和STOS串操作指令配合使用。

2）REPE/REPZ　有条件重复前缀。

- 指令功能：该指令条件为相等/结果为0时重复前缀，该前缀与CMPS和SCAS串操作指令配合使用。
- 只有当CX≠0且ZF=1（表示两个操作数比较相等）时，继续执行其后的比较或扫描指令；否则当CX=0或者ZF=0（表示两个操作数比较不相等）时，则停止执行其后的字符串指令，结束该操作。

3） REPNE/REPNZ

有条件重复前缀。

指令功能：该指令条件为不相等/结果不为0时重复前缀。

- 有关串操作指令的重复前缀、操作数以及地址指针所用的寄存器等情况归纳如下，见表3.3所示。

指令	重复前缀	操作数	地址指针寄存器
MOVS	REP	目标	ES:DI
		源	DS:SI
CMPS	REPE/REPZ	目标	DS:SI
		源	ES:DI
SCAS	REPE/REPZ	目标	ES:DI
LODS	无	源	DS:DI
STOS	REP	目标	ES:DI

80386串操作指令

位串操作指令

- 串操作指令包括 MOVS、CMPS、SCAS、LODS、STOS、INS、OUTS。
- 前5个指令是8086中原有的，在80386指令系统增加了对32位寄存器的支持，如MOVS可以写成MOVSB、MOVSW、MOVSD。它们的源变址寄存器、目的变址寄存器及计数器是用ESI、EDI、ECX还是用SI、DI、CX由它所在的段决定。若是32位段，则使用前者，否则使用后者。

串操作程序设计举例

例

```
S_POINT   DD S_ADDR
D_POINT   DD D_ADDR
S_ADDR    DB…                          ; 源数据区
 ┆
D_ADDR    DB…                          ; 目的数据区
 ┆
LDS   SI, S_POINT                       ; 置全地址指针DS：SI值
LES   DI, D_POINT                       ; 置全地址指针ES：DI值
    CLD                                     ; 将方向标志DF清0
MOV   CX, LENGTH                    ; 置串长度值
REP   MOVSB
```

例

把DATA1段从0020H开始的30H个字节串和DATA2段从0100H开始的30H个字节串进行比较。

解：依题意，程序段为：

```
MOV   AX, DATA1
MOV   DS, AX
MOV   AX, DATA2
MOV   ES, AX
MOV   SI, 0020H
MOV   DI, 0100H
MOV   CX, 0030H
CLD
REPE  CMPS
```

例 编写程序查找STR1串中是否有字母‘J’，如果有输出‘Y’，否则输出‘N’。

```
DATAS SEGMENT
STR1 DB ‘ASDFGHJK’
DATAS ENDS
CODES SEGMENT
    ASSUME CS:CODES,DS:DATAS,ES:
DATAS
START:
    MOV AX,DATAS
    MOV DS,AX
    MOV ES,AX
MOV DI,OFFSET SRT1
MOV AL,’ J’
MOV CX,8
 REPNZ SCASB;字符搜索
JNZ  JM1
MOV DL,’ Y’
MOV AH,2
    INT 21H
    JMP EXT
JM1:MOV DL,’ N’
    MOV AH,2
    INT 21H
EXT:MOV AH,4CH
    INT 21H
CODES ENDS
    END START
```

段的概念和段寄存器

- https://www.bilibili.com/video/BV1jX4y1j7Fz/?spm_id_from=333.788.recommend_more_video.0

- https://www.bilibili.com/video/BV1rP411x7Nr/

https://www.bilibili.com/video/BV1J34y1h797/?p=30&vd_source=3a145caaf017ae25333027a8601be04c



子程序设计

PART 03

- 当程序功能相对复杂，所有的语句序列均写到一起时，程序结构将显得零乱
- 编程时把功能相对独立的程序段单独编写和调试，作为一个相对独立的模块供程序使用，这就是子程序。
- 子程序可以实现源程序的模块化，简化源程序结构。

- 子程序还可以使模块得到复用，进而提高编程效率。

子程序的定义与调用

1. 子程序的定义

过程名　PROC [NEAR/FAR]

过程体

过程名　ENDP

2. 过程的调用与返回

- 由指令CALL和RET来完成
- 要保证RET指令执行前堆栈栈顶的内容刚好是过程返回的地址
- 注意寄存器的保护与恢复

子程序调用与返回指令

（1）调用指令CALL

用于调用子程序。

1）段内直接调用

指令格式：CALL　DST

要求：DST为子程序名。

指令功能：调用一个近子程序，该子程序在本段内。指令汇编后，得到CALL的下一条指令与被调用的子程序入口地址之间的16位相对偏移量DISP。段内调用指令功能是将当前指令指针IP内容自动压入堆栈，堆栈指针SP自动减2，然后将相对偏移量DISP送到IP中，使程序控制转移到调用的子程序。

2）段内间接调用

指令格式：CALL　SRC

要求：SRC为16位寄存器或者各种寻址方式的存储器操作数。

指令功能：将指令指针寄存器IP的内容自动压入堆栈，然后将SRC的内容传送到IP，转到子程序的入口地址去执行。

例 CALL　CX　　；子程序入口地址的偏移量为CX的内容

CALL　WORD PTR [BX+SI+20H]　　；子程序入口地址：IP←[BX+SI+20H]单元中的字数据

3）段间直接调用

指令格式：CALL　DST

要求：DST给出子程序入口地址的完整信息，即段基地址CS和偏移量IP。

指令功能：把现行CS和IP值压入堆栈，再把子程序对应段基址和偏移地址分别送入CS和IP，程序转向子程序执行。

例 CALL　2500H:1000H　　；子程序入口地址为2500H:1000H

CALL　FAR PTR PROCNAME　　；子程序入口地址在另一段内

例 假设在存储器中存有两个目标子程序地址 SIP0和SIP1，序号分别为0，1，下面一段程序根据SI寄存器的赋值，能调用不同目标子程序。

```
PROCTABL   DD   SIP0
DD   SIP1
      ┆
MOV   AX, SEG   PROCTABL
MOV   DS, AX
MOV   SI, ENTRY
; ENTRY是目标子程序的序号的4倍，
; 即相对于PROCTABL的字节地址
CALL   FAR PTR PROCTABL [SI]
```

（2）返回指令RET

1）段内返回

指令格式：RET

指令功能：把保护堆栈区的断点偏移地址送入指令指针寄存器IP，返回调用程序的地方继续执行。

2）段间返回

指令格式：RET

指令功能：把保护堆栈区的断点偏移地址送入指令指针寄存器IP，段地址送到CS寄存器，返回调用程序的地方继续执行。使用该指令时，应注意保证CALL指令的类型与子程序中RET指令的类型匹配，以免发生返回地址错误。如果在定义子程序中，该子程序定义为远子程序FAR，对应的返回指令RET属于段间返回，若子程序定义为近子程序NEAR，对应的返回指令RET属于段内返回。

3）带参数返回

指令格式：RET n

注意：n为常数或表达式，n只能是16位偶数，不能为奇数。

该指令也分为段内（近子程序）返回和段间（远子程序）返回，其操作与返回指令RET基本相同，不同之处是最后将修改堆栈指针SP，即SP←SP+n。

3．子程序设计举例

例　编制一个过程把AL寄存器内的二进制数用十六进制形式在屏幕上显示出来。

分析：AL中8位二进制数对应2位十六进制数，先转换高4位成ASCII码并显示，然后转换低4位并显示。屏幕显示采用02号DOS功能调用。

子程序的参数传递

- 主程序在调用子程序时，通常需要向其提供一些数据，对于子程序来说就是入口参数（输入参数）
- 子程序执行结束也要返回给主程序必要的数据，这就是子程序的出口参数（输出参数）。

1．用寄存器传递参数

2．用变量传递参数

3．用堆栈传递参数

1．用寄存器传递参数

例 设ARRAY是10个元素的数组，每个元素是8位数据。试用子程序计算数组元素的校验和，并将结果存入变量RESULT中。

分析：子程序完成元素求和，主程序需要向它提供入口参数，使得子程序能够访问数组元素。子程序需要回送求和结果这个出口参数。本例我们采用寄存器传递参数。

主程序

```
.MODEL      SMALL
.STACK
.DATA
    COUNT   EQU   10              ; 数组元素个数
    ARRAY   DB 12H, 25H, 0F0H, 0A3H, 3, 68H, 71H
            DB 0CAH, 0FFH, 90H    ; 数组
    RESULT  DB    ?               ; 校验和
.CODE
.STARTUP     ; 设置入口参数（含有DS←数组的段地址）
     MOV    BX, OFFSET ARRAY  ; BX←数组的偏移地址
     MOV    CX, COUNT         ; CX←数组的元素个数
     CALL   CHECKSUMA         ; 调用求和过程
     MOV    RESULT, AL        ; 处理出口参数
    .EXIT 0
```

- 子程序

```
CHECKSUMA  PROC
           XOR       AL, AL       ; 累加器清0
SUMA:  ADD   AL, [BX]; 求和
           INC       BX           ; 指向下一个字节
           LOOP SUMA
           RET
CHECKSUMA      ENDP
END
```

2．用变量传递参数（同上例）

用COUNT，ARRAY和RESULT变量。主程序只要设置数据段DS，就可以调用子程序；子程序直接采用变量名存取数组元素。

```
.MODEL    SMALL
.STACK
.DATA
  COUNT   EQU   10            ; 数组元素个数
  ARRAY   DB 12H, 25H, 0F0H, 0A3H, 3, 68H, 71H
          DB 0CAH, 0FFH, 90H        ; 数组
  RESULT  DB    ?                   ; 校验和
.CODE
.STARTUP                            ; 含有DS←数组的段地址
    CALL  CHECKSUMB    ; 调用求和过程
.EXIT 0
```

```
CHECKSUMB       PROC
                PUSH    AX
                PUSH    BX
                PUSH    CX
                XOR     AL, AL                  ; 累加器清0
                MOV     BX, OFFSET ARRAY        ; BX←数组的偏移地
                MOV     CX, COUNT           ; CX←数组的元素个数
          SUMB: ADD     AL, [BX]                ; 求和
                INC             BX
                LOOP    SUMB
                MOV     RESULT, AL              ; 保存校验和
                POP     CX
                POP     BX
                POP     AX
                RET
CHECKSUMB   ENDP
    END
```

3．用堆栈传递参数（同上例）

- 通过堆栈传递参数，主程序将数组的偏移地址和元素个数压入堆栈，然后调用子程序；
- 子程序通过BP寄存器从堆栈相应位置取出参数（非栈顶数据），求和后用AL返回结果。
- 由于共用数据段，所以没有传递数据段基地址。本例利用堆栈传递入口参数，但出口参数仍利用寄存器传递。

```
.MODEL  SMALL
    .STACK
    .DATA
    COUNT               EQU 10                      ; 数组元素个数
    ARRAY       DB 12H, 25H, 0F0H, 0A3H, 3, 68H, 71H, 0CAH, 0FFH,
    90H ; 数组
    RESULT      DB      ?                           ; 校验和
    .CODE
    .STARTUP
            MOV         AX, OFFSET ARRAY            ; 设置入口参数
            PUSH        AX                          ; 压入数组的偏移地
    址
            MOV         AX, COUNT
         PUSH   AX                              ; 压入数组的元素个数
            CALL        CHECKSUMC               ; 调用求和过程
            ADD         SP, 4                   ; 主程序平衡堆栈
            MOV         RESULT, AL                  ; 保存校验和
           .EXIT 0
```

```
CHECKSUMC      PROC
          PUSH  BP
          MOV   BP, SP    ; BP指向当前栈顶，用于取出入口参数
          PUSH  BX        ; 保护使用的BX和CX寄存器
          PUSH  CX
          MOV   BX, [BP+6]  ; BX←SS: [BP+6] (数组的偏移地址)
          MOV   CX, [BP+4]  ; CX←SS: [BP+4] (数组的元素个数)
          XOR   AL, AL          ; 累加器清0
     SUMC: ADD  AL, [BX]        ; 求和: AL← (AL)  + DS: [BX]
          INC   BX
          LOOP  SUMC
          POP   CX              ; 恢复寄存器
          POP   BX
          POP   BP
          RET
     CHECKSUMC  ENDP
                END
```

子程序的嵌套、递归与重入

- **1．子程序的嵌套**
  - 子程序内包含有子程序的调用就是子程序嵌套。
  - 嵌套深度（即嵌套的层次数）逻辑上没有限制，
  - 由于子程序的调用需要在堆栈中保存返回地址以及寄存器等数据，因此实际上受限于开设的堆栈空间。

我们可以写成过程，形成过程（子程序）嵌套：

```
ALDISP  PROC                    ; 显示AL中的2位十六进制数
        PUSH   AX                       ; 保护入口参数
        PUSH   CX
        PUSH   AX                       ; 暂存数据
        MOV    CL, 4
        SHR          AL, CL; 转换AL的高4位
        CALL   HTOASC          ; 子程序调用（嵌套）
        POP    AX                       ; 转换AL的低4位
        CALL   HTOASC          ; 子程序调用（嵌套）
        POP          CX
        POP          AX
        RET                             ; 子程序返回
ALDISP          ENDP
```

```
HTOASC    PROC ; 将AL低4位表达的一位十六进制数转换为ASCII码
          PUSH       AX   ; 保护入口参数
          PUSH       BX
          PUSH       DX
          MOV BX, OFFSET ASCII       ; BX指向ASCII码表
          AND AL, 0FH                ; 取得一位十六进制数
          XLAT       ES: ASCII ; 换码: AL←CS: [BX+AL]
      MOV     DL, AL ; 显示
          MOV AH, 2
          INT 21H
          POP DX
         POP  BX
         POP  AX
         RET
ASCII  DB 30H, 31H, 32H, 33H, 34H. 35H, 36H, 37H, 38H, 39H
       DB 41H, 42H, 43H, 44H, 45H, 46H; 子程序的数据区
 HTOASC  ENDP
```

2．子程序的递归

- 子程序直接或间接地嵌套调用自身时称为递归调用
- 含有递归调用的子程序称为递归子程序。

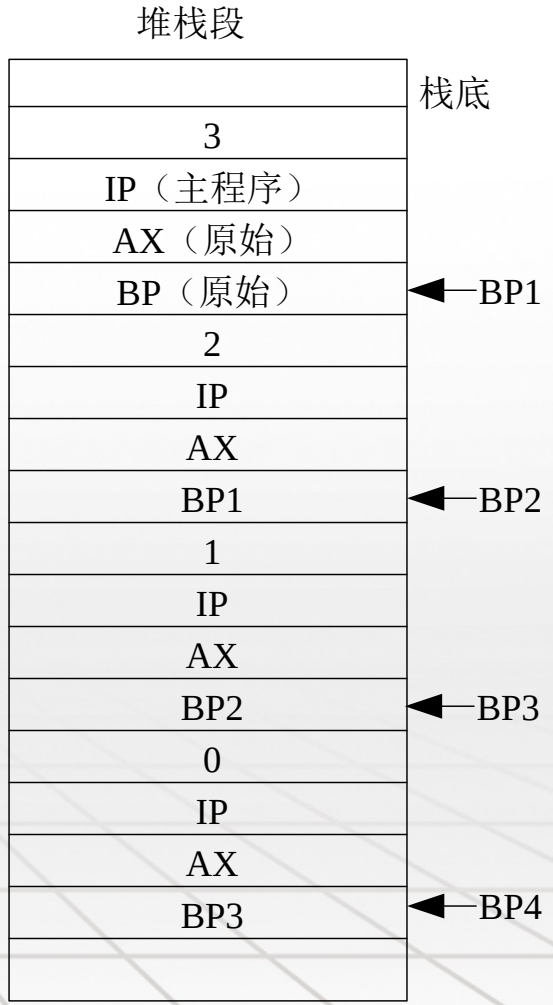
- 递归子程序必须采用寄存器或堆栈传递参数，递归深度受堆栈空间的限制。

例 编制计算N! =N × (N−1) × (N−2) × …× 2 × 1 (N≥0) 的程序

分析：已知递归定义

$$N! = \begin{cases} N \times (N-1)! & N>0 \\ 1 & N=0 \end{cases}$$

- 求N!可以设计成输入参数为N的递归子程序
- 每次递归调用的输入参数递减1。
- 如果N>0，则由当前参数N乘以递归子程序返回值得到本层返回值；
- 如果递归参数N＝0，得到返回值为1



3．子程序的重入

- 子程序的重入是指子程序被中断后又被中断服务程序所调用
- 能够重入的子程序称为可重入子程序。
- 在子程序中，注意利用寄存器和堆栈传递参数和存放临时数据，而不要使用固定的存储单元（变量），就能够实现重入。

子程序的应用

例 从键盘输入有符号的10进制数的子程序。

分析：子程序从键盘输入一个有符号的10进制数。负数用“-”引导，正数直接输入或用“+”引导。子程序还包含将ASCII码转换为二进制数的过程，其算法如下：

①首先判断输入正数还是负数，并用一个寄存器记录下来；

②接着输入0~9数字（ASCII码），并减30H转换为二进制数；

③然后将前面输入的数值乘10，并与刚输入的数字相加得到新的数值；

④重复②、③步，直到输入一个非数字字符结束；

⑤如果是负数进行求补，转换成补码；否则直接将数值保存。

例 向显示器输出有符号十进制数的子程序。

分析：子程序在屏幕上显示一个有符号十进制数，负数用“-”引导。子程序还包含将二进制数转换为ASCII码的过程，其算法如下：

① 首先判断数据是零、正数或负数，是零显示“0”并退出；

② 是负数，显示“-”，求数据的绝对值；

③ 接着数据除以10，余数加30H转换为ASCII码压入堆栈；

④ 重复③步，直到余数为0结束；

⑤ 依次从堆栈弹出各位数字，进行显示。



高级汇编语言程序设计

PART 04

- MASM 6.0开始引入高级语言具有的程序设计特性:
  - 即分支和循环的流程控制伪指令,
  - 带参数的过程定义、声明和调用伪指令,

条件控制伪指令

MASM 6.0引入.IF, .ELSEIF, .ELSE和.ENDIF伪指令

```
.IF条件表达式
    分支体1
    [.ELSEIF条件表达式

    分支体2]
    ---
    [.ELSE

    分支体N]
    .ENDIF
```

例如，求AX绝对值的单分支结构程序。

```
.IF             AX<0                    ; 等价于.IF SIGN?
                NEG    AX               ; 满足，求补
.ENDIF
                MOV    RESULT, AX
```

```
. IF          AX = = 5
              MOV   BX, AX
              MOV   AX, 0
.ELSE
              DEC   AX
.ENDIF
```

LST文件中，汇编以后代码如下:

```
.IF             AX  = = 5
    *                   CMP     AX, 05H
    *                   JNE             @C0001
                        MOV     BX, AX
                        MOV     AX, 0
.ELSE
    *                   JMP             @C0003
    *@C0001:            DEC     AX
    .ENDIF
    *@C0003:
```

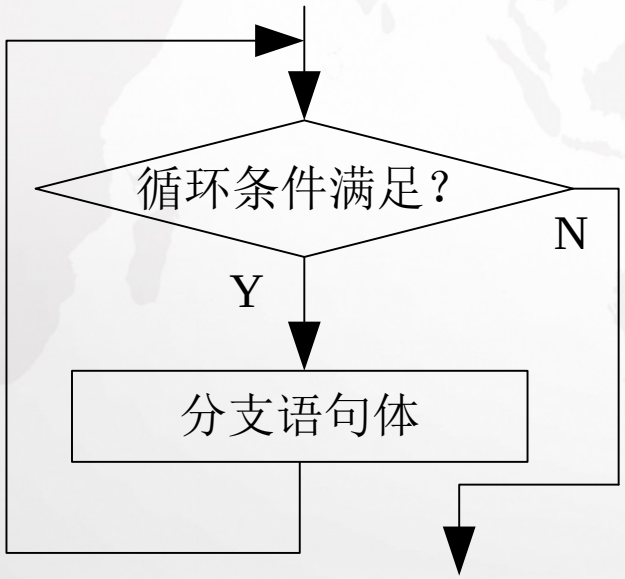
循环控制伪指令

1．WHILE结构的循环控制伪指令

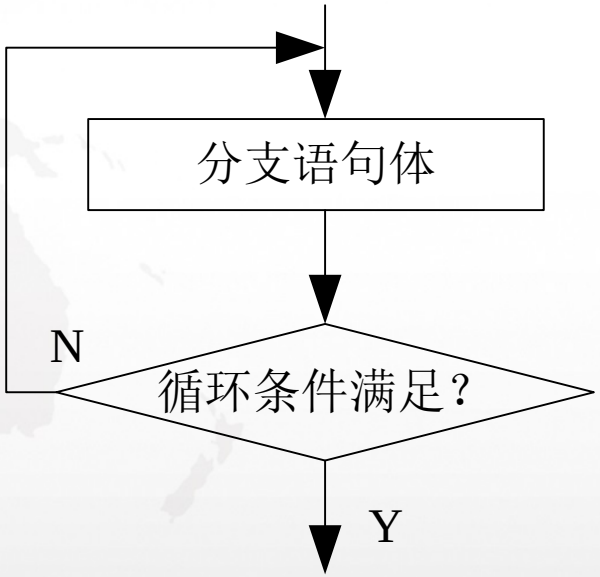
```
.WHILE条件表达式
        循环体
.ENDW
```

2．UNTIL结构的循环控制伪指令

```
.REPEAT                    ；重复执行循环体
        循环体
.UNTIL条件表达式    ；直到条件为真
```



A）WHILE 循环结构



B）UNTIL 循环结构

过程声明和过程调用伪指令

NEAR
/FAR

任何有效的类型

1．过程定义伪指令

在MASM 6．x中，带有参数的过程定义伪指令格式如下：

```
过程名 PROC  [调用距离][语言类型][          ]
                [<起始参数>][USES寄存器列表]
                [, 参数：[类型]]．．．
             LOCAL参数表
             ．．．          ；汇编语言语句
过程名  ENDP
```

作用范围

- 默认是PUBLIC，表示其它模块可见
- PRIVATE表示对外不可见；
- EXPORT隐含有PUBLIC和FAR，表示该过程应该放置在导出表（Export entry table）中

2. 过程局部变量定义伪指令

- LOCAL变量名[个数] [: 类型] [, . . .]

3．过程声明伪指令

PROTO是一个过程声明伪指令，用于事先声明过程的结构。

过程名 PROTO［调用距离］［语言类型］［，参数：［类型］］

4. INVOKE过程调用伪指令

- CALL是进行子程序调用的硬指令。
- 与PROTO配合使用的过程调用伪指令是INVOKE，它的格式如下：
  - INVOKE　　过程名[，参数，．．．]



汇编语言与C语言混合编程

PART 13

C程序嵌入汇编

C程序内嵌入汇编指令的方法简单明快，不必考虑两者间的接口,省去了独立汇编和连接的步骤,从而使用更加方便灵活,在内嵌汇编语句中可以通过名字引用C语言有效的标号、常量、变量名、函数名、宏等信息。

（1）在嵌入的汇编指令前必须用关键字asm。

说明asm <操作码><操作数><;或换行>。

其中，操作码是有效的80X86指令及某些汇编伪指令，如db ，dw， dd及extern。操作数可以是C语言中的常量、变量和标号， 也可以是操作码可接受的数。内嵌的汇编指令用分号或换行作为结束。需要注意的是不能像MASM中那样用用分号作为注释的开始，必须用C方式即/*… …*/来标出注释行。一条汇编指令不能超越两行， 但同一行中可有多条汇编指令。

例4.52

```
asm  mov ax,ds ; /* ds-> ax * /
asm  push ds ;
```

或者用换行结束也可以， 如下:

```
asm mov ax,ds
asm push ds
```

（2）内嵌汇编指令中的操作数

内嵌汇编指令的操作数可以是C语言程序中的符号，编译时它会自动转换为适当的操作数。当汇编指令中使用寄存器名时,只能是80X86的寄存器名，并且不分大小写。另外，在C语言中，DI和SI两个寄存器常被用来存放Register变量，只有在C程序中没有指定Register变量时，内嵌汇编程序才可以任意使用这两个寄存器。

(3) 汇编指令操作数可以是结构数据。

例4.53

```
struct student
{
 int num ;
 char name[10] ;
}stu;
```

在汇编指令中可以用结构体的成员作为操作数

```
asm mov ax,stu.num
```

（4）转移指令的执行。

内嵌汇编指令可以使用任何有条件和无条件转移汇编指令，他们只能在函数体内有效，不允许段间转移。由于在asm语句中无法给出标号，所以转移指令只能使用C语言中的goto语句用的标号。

例4.54

```
asm jmp exitfunc
        ……
exitfunc;
return 1;
```

(5)程序编译

C程序中内嵌汇编指令时，编译连接只能用TCC命令行，而且必须有_B选项。

注意:

（1）C 程序调用汇编中的寄存器时，要用大写且前面加一个下划线，如上例中return(_AX)。

（2）在编译之前从TC下include文件夹中把stdio.h和stdarg.h复制到TC目录下，把lib文件夹中的cs.obj,cs.lib,emu.lib和maths.lib复制到TC目录下。

（3）若TC文件夹中没有TASM.exe可以把微软的MASM.exe复制到TC目录下并且改名为TASM.exe。

（4）编泽时只能在纯DOS方式下(不是TC的SHELL os)，用TC中的TCC进行编译链接。编译时在命令行键入：C:\TC\TCC _B asm.c。可以生成文件asm.bak、asm.obj、asm.map、asm.exe。其中asm.exe是得到的可执行文件。

Microsoft C程序内嵌汇编指令方法。VC++6.0是微软公司为开发的32位应用程序而推出的基于C\C++的集成开发环境。它支持内嵌汇编指令的混合编程，并且VC++的编译器中已经集成了汇编指令编译器，使得编译连接非常方便。实现步骤如下：

（1）在VC++中启用汇编语句的关键字是_asm。

（2）_asm不能单独出现，后面必须有汇编指令或者是由大括号括起来的多条指令。

（3）在_asm所带的一组汇编指令中可以有标号，C\ C++中的goto语句和汇编指令的跳转语句可跳到汇编指令组中的标号处,也可以跳转到汇编指令组外的标号处。

(4)在_asm 所带的汇编语块中只能调用没有重载的全局的C++函数，也可以调用声明为exetrn C类型的函数。因为C标准库的函数全部声明为这种形式，所以C标准库函数都可以调用。

（5）对于类、结构体和共用体的成员变量，汇编指令可以直接使用。

汇编语言与C语言模块连接

1.汇编语言与C语言模块结构

Microsoft C和Turbo C编译系统将不同类型的变量保存在不同段内，其具体情况如下：
BSS 段——存放未初始化的静态变量
DATA 段——存放所有的全局变量和已初始化的静态变量
CONST 段——存放只读常数
STACK 段—— 存放自动变量和函数参数
TEXT 段—— 存放程序的执行代码而且，经常将DATA ， CONST ，BSS 和STACK 段组合成一个DGROUP 段组。

C 编译系统对不同的段的段名、边界类型、结合类型及类型规定了统一的命名规则

段名	边界类型	结合类型	类型
TEXT	BYTE	PUBLIC	‘CODE’
DATA	WROD	PUBLIC	‘DATA’
BSS	WORD	PUBLIC	‘BSS’
CONST	WROD	PUBLIC	‘CONST’
STACK	PARA	STACK	‘STACK’

2. 调用约定

（1）标识符的约定
（2）参数传递约定
（3）寄存器使用约定

3. 编程方法

（1）C程序调用汇编语言程序

例4.58下面是在小模式下的混合编程例子。该文件有min. c 和min_num. asm 两个部分组成。功能为从5个数中找出最小的。min. c 是主程序，由C 语言编写。min_num. asm 是汇编子程序，完成寻找小数的工作。由min. c 来调用它。程序代码如下:

```
min. c
int extern min_num (int count , int x1, int x2, int x3,
int x4, int x5) ;
main ()
{int i;
i= min_num (5,7,0,6,1,12 ) ;
printf (“The mininum of five is % d\n”, i) ;
}

min_num. asm
TEXT  SEGMENT  BYTE  PUBLIC ‘CODE ’
PUBLIC  min_num
min_num  proc  far
     ASSUME CS: TEXT
push bp
mov bp , sp
mov ax,0
mov cx,[ bp+4]
cmp cx , ax
jle exit
mov ax,[ bp+6]
jmp ltest
comp: cmp ax ,[ bp +6]
          jle ltest
          mov ax,[ bp+6]
ltest: add bp,2
          loop comp
exit: pop bp
          ret
min_num endp
TEXT ENDS
               end
```

（2）汇编语言程序调用C函数